

Analysis Report

Like

Share

839 people like this. [Sign Up](#) to see what your friends like.

↻ Share this Report

GC log file: **gc-2017_07_28-11_58.log.0**

Duration: **1 hr 10 sec**

🕒 Application waiting for resources ⚠️

It looks like your application is waiting due to lack of compute resources (either CPU or I/O cycles). Serious production applications shouldn't be stranded because of compute resources. In **24 GC event(s)**, 'real' time took more than 'usr' + 'sys' time.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2017-07-28T12:12:01	0.45	0.05	1.59
2017-07-28T12:12:09	0.51	0.02	1.98
2017-07-28T12:16:38	0.51	0.01	0.72
2017-07-28T12:16:49	0.41	0.02	2.16
2017-07-28T12:19:12	0.58	0.05	1.36
2017-07-28T12:19:18	0.79	0.06	3.06
2017-07-28T12:19:32	0.56	0.08	4.01
2017-07-28T12:20:22	0.35	0.01	0.9
2017-07-28T12:21:24	0.48	0.01	1.17
2017-07-28T12:21:32	0.51	0.02	0.99
2017-07-28T12:21:45	0.33	0.02	3.92
2017-07-28T12:22:43	0.74	0.03	1.2
2017-07-28T12:22:48	0.46	0.02	1.51
2017-07-28T12:22:58	0.93	0.02	1.46
2017-07-28T12:25:45	0.6	0.03	1.97
2017-07-28T12:25:53	0.54	0.05	3.46
2017-07-28T12:32:51	0.66	0.03	2.72
2017-07-28T12:35:53	0.67	0.01	1.06
2017-07-28T12:36:18	1.43	0.02	2.37
2017-07-28T12:37:27	0.41	0.01	0.64
2017-07-28T12:42:44	0.62	0.01	1.45
4 more events...			

Read our recommendations to [fix this problem \(https://blog.gceasy.io/2016/12/08/real-time-greater-than-user-and-sys-time/\)](https://blog.gceasy.io/2016/12/08/real-time-greater-than-user-and-sys-time/)

🕒 System Time greater than User Time ⚠️

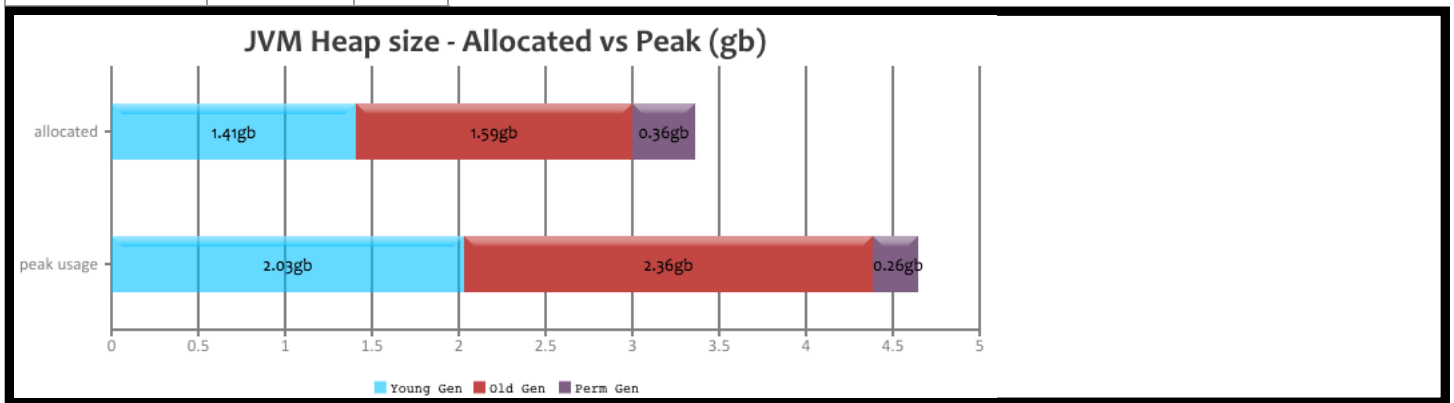
In **1 GC event(s)**, 'sys' time is greater than 'usr' time. It's not a healthy sign.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2017-07-28T11:59:59	0.53	1.44	0.56

Read our recommendations to [reduce sys time \(https://blog.gceasy.io/2016/12/11/sys-time-greater-than-user-time/\)](https://blog.gceasy.io/2016/12/11/sys-time-greater-than-user-time/)

📊 JVM Heap Size

Generation	Allocated 📄	Peak 📄
Young Generation	1.41 gb	2.03 gb
Old Generation	1.59 gb	2.36 gb
Perm Generation	372 mb	270 mb
Young + Old + Perm	3.36 gb	2.98 gb



🔍 Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here \(https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/\)](https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

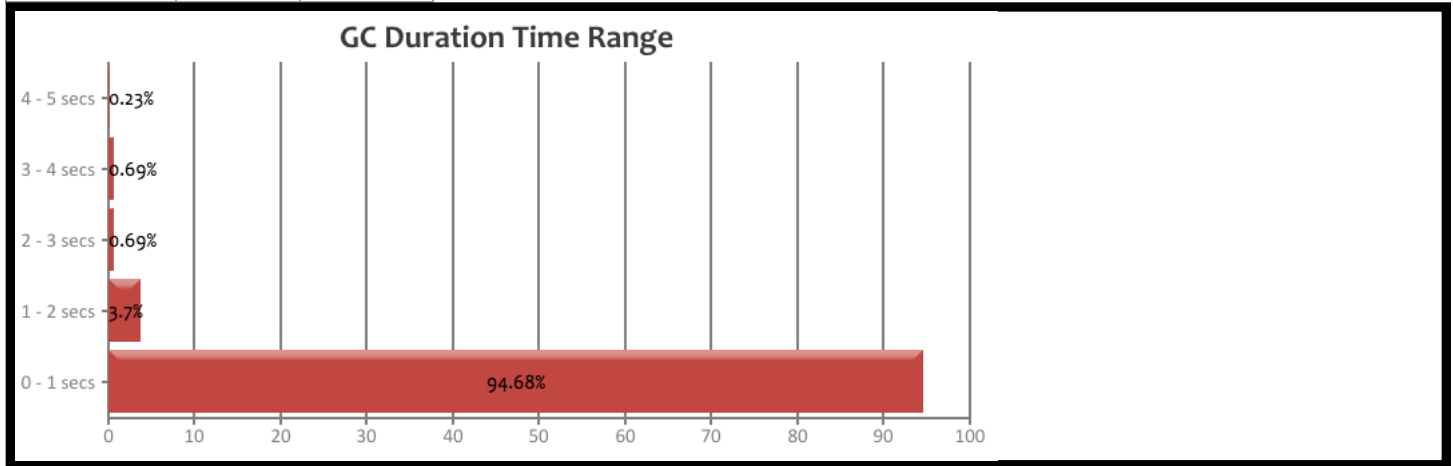
1 Throughput 📄 : 96.07%

2 Latency:

Avg GC Time 📄	308 ms
Max GC Time 📄	4 sec 10 ms

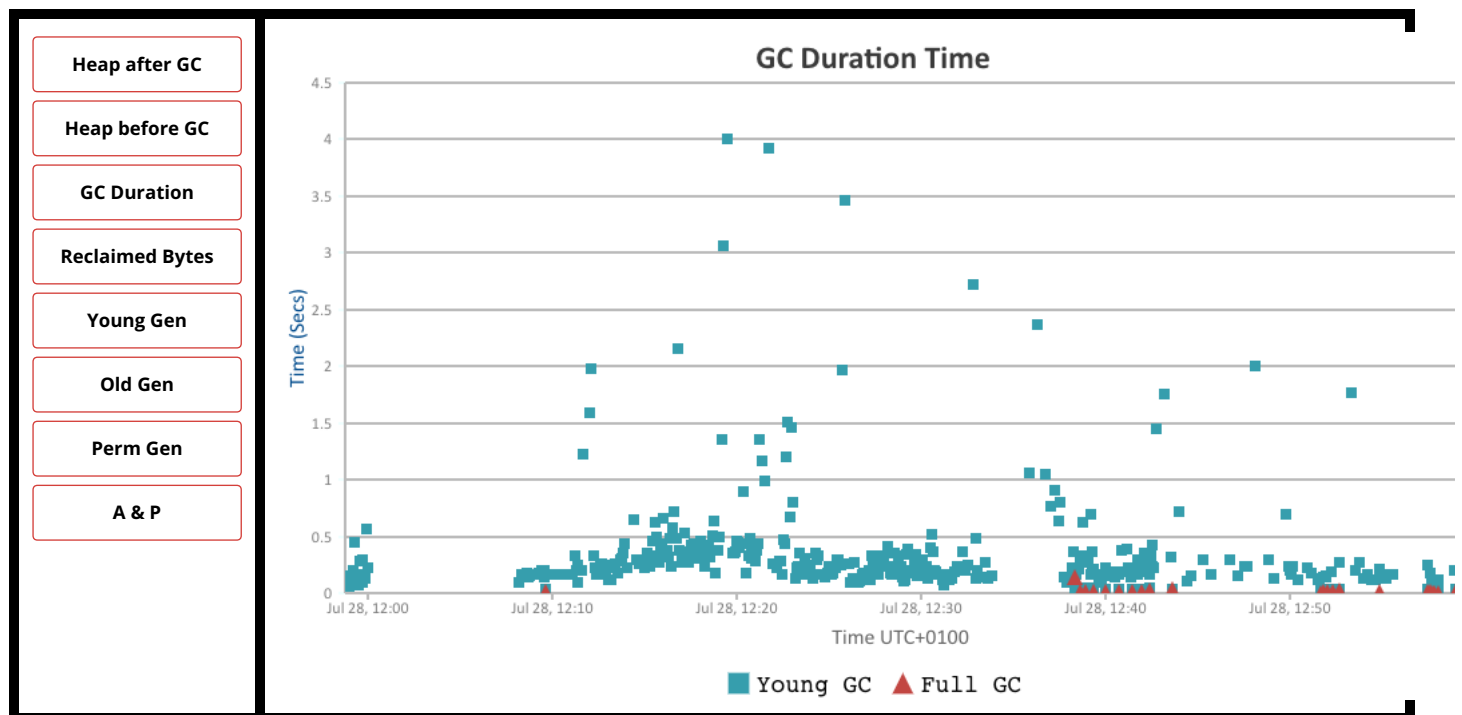
GC Duration Time Range 📄:

Duration (secs)	No. of GCs	Percentage
0 - 1	409	94.676%
1 - 2	16	98.38%
2 - 3	3	99.074%
3 - 4	3	99.769%
4 - 5	1	100.0%



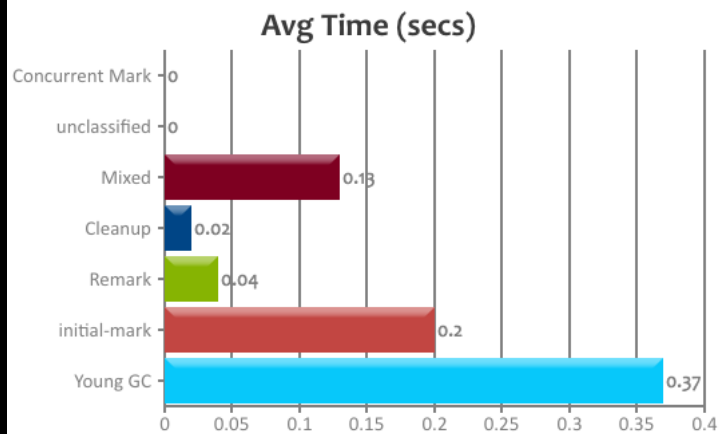
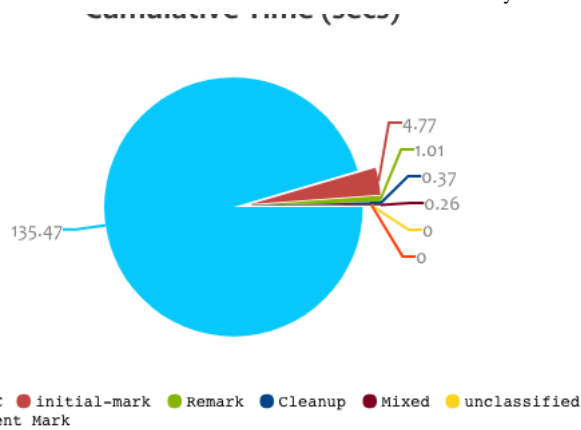
Interactive Graphs

(All graphs are zoomable)



Ⓞ G1 GC Statistics

Cumulative Time (secs)



	unclassified	Concurrent Mark	Remark	Young GC	Cleanup	Mixed	initial-mark	Total
Count ?	1	23	23	364	23	2	24	460
Total GC Time ?	0	0	1 sec 10 ms	2 min 15 sec 470 ms	370 ms	260 ms	4 sec 770 ms	2 min 21 sec 880 ms
Avg GC Time ?	0	0	44 ms	372 ms	16 ms	130 ms	199 ms	308 ms
Avg Time std dev	0	0	19 ms	494 ms	28 ms	20 ms	70 ms	458 ms
Min/Max Time ?	0 / 0	0 / 0	0 / 130 ms	0 / 4 sec 10 ms	0 / 140 ms	0 / 150 ms	0 / 360 ms	0 / 4 sec 10 ms
Avg Interval Time ?	n/a	2 min 14 sec 595 ms	2 min 14 sec 594 ms	9 sec 388 ms	2 min 14 sec 595 ms	14 sec 511 ms	2 min 8 sec 788 ms	33 sec 704 ms

🔧 Object Stats

(These are good metrics to include in your performance reports)

Total created bytes ?	241.79 gb
Total promoted bytes ?	2.1 gb
Avg creation rate ?	68.58 mb/sec
Avg promotion rate ?	609 kb/sec

💧 Memory Leak ⓘ

No major memory leaks.

(**Note:** there are [8 flavours of OutOfMemoryErrors](https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf) (https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf). With GC Logs you can diagnose only 5 flavours of them (Java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

📄 Consecutive Full GC ⓘ

None.

📄 Long Pause ⓘ

None.

🕒 Safe Point Duration ⓘ

(To learn more about SafePoint duration, [click here](https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/) (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

❓ GC Causes ⓘ

(What events caused the GCs, how much time it consumed?)

Not reported in the log.

🔄 Tenuring Summary ⓘ

Not reported in the log.

📄 Command Line Flags ⓘ

Not reported in the log.

🏆 Become a DevOps champion in your organization

(Best practises/tools)

- ✔ Use **fastThread.io** (https://fastthread.io/) to analyze thread dumps, core dumps and hs_err_pid files
- ✔ Do proactive Garbage Collection analysis on all your JVMs (not just one or two) [using the GC log analysis API](https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/) 🌟 (https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/)